

# Reviewer Pack — Minimal Rank of Quotient Singularities over $AC^0$ Parity Circu...

Entry b94df982d33d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 23:43:03 UTC

## Minimal Rank of Quotient Singularities over $AC^0$ Parity Circuit Depth

**Entry ID:** b94df982d33d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 23:43:03 UTC

### 1. Conjecture

**Field A** (mathematical branch): Algebraic Geometry (Quotient Singularities) **Field B** (complexity object): Complexity Theory:  $AC^0$  Parity Circuit Complexity

#### Statement:

[‘For any polynomial-size  $AC^0$  parity circuit  $C$  with depth  $d$ , the minimal rank of its associated quotient singularity is  $\Theta(d^2 \log n)$ .’, ‘Equivalently, for any constant  $c > 0$  and any  $n$ -variable 3-CNF formula  $\varphi$ , if there exists an  $AC^0$  parity circuit  $C$  computing  $\varphi$  with depth less than  $cn^{1/2}$ , then the minimal rank of the quotient singularity associated with  $C$  is greater than  $d^2 \log n$ .’, ‘Furthermore, for all instances of size  $n \leq 40$ , this conjecture holds with high probability.’]

#### Rationale (proposer’s reasoning):

[‘Quotient singularities provide a rich algebraic-geometric structure that could potentially encode the complexity of computational tasks such as those performed by  $AC^0$  parity circuits.’, ‘Previous work has established connections between algebraic geometry and circuit complexity, suggesting that quotient singularities may play a crucial role in understanding the lower bounds for  $AC^0$  parity circuits.’, ‘This conjecture aims to explore this connection by proposing a quantitative relationship between the minimal rank of quotient singularities and the depth of  $AC^0$  parity circuits.’]

**Taxonomy category:**  $AC0\_PARITY$  (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 07bd95e70a92b820

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

For each  $AC^0$  parity circuit  $C$ , if its associated quotient singularity has a minimal rank greater than  $d^2 \log n$  for all seeds and the mean metric across seeds is  $\leq 3$ , then the conjecture is supported. If any seed produces a metric  $> 10$  or the mean metric across seeds  $> 3$ , then the conjecture is falsified.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "quotient singularity" AND "AC<sup>0</sup> parity circuit depth" - "algebraic geometry" AND "complexity theory" AND "polynomial-size AC<sup>0</sup> parity circuit" - "minimal rank" IN "quotient singularities" AND "depth d" IN "AC<sup>0</sup> parity circuit complexity"

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, clause_density):
        num_clauses = int(n * clause_density)
        clauses = []
        for _ in range(num_clauses):
            literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            random.shuffle(literals)
            clauses.append(' '.join(map(str, literals)))
        return ' '.join(clauses)

    def ac0_parity_circuit(depth, n):
        if depth == 0:
            return [random.choice([1, -1]) * (i + 1) for i in range(n)]
        else:
            inputs = ac0_parity_circuit(depth - 1, n)
            outputs = []
            for i in range(n):
                output = 1
                for j in range(i, n, 2):
                    output *= inputs[j]
                outputs.append(output)
            return outputs

    def gaussian_elimination(matrix):
```

```

rows, cols = len(matrix), len(matrix[0])
rank = 0
for i in range(cols):
    if rank < rows:
        pivot_row = rank
        while matrix[pivot_row][i] == 0:
            pivot_row += 1
            if pivot_row == rows:
                break
        if pivot_row != rank:
            matrix[rank], matrix[pivot_row] = matrix[pivot_row], matrix[rank]
        for j in range(rank + 1, rows):
            factor = -matrix[j][i] / matrix[rank][i]
            for k in range(i, cols):
                matrix[j][k] += factor * matrix[rank][k]
        rank += 1
return rank

def minimal_rank_of_quotient_singularity(n, depth):
    circuit = ac0_parity_circuit(depth, n)
    matrix = [[circuit[i]] for i in range(n)]
    return gaussian_elimination(matrix)

n_values = [10, 20, 30, 40]
d_values = [int(c * n**0.5) for c in [0.1, 0.2, 0.3, 0.4]]
results = []

for n in n_values:
    for d in d_values:
        if d > n**0.5:
            continue
        clause_density = 0.5
        formula = generate_3cnf(n, clause_density)
        rank = minimal_rank_of_quotient_singularity(n, d)
        results.append({
            "n": n,
            "d": d,
            "rank": rank,
            "expected": d**2 * math.log(n),
            "conjecture_holds": rank > d**2 * math.log(n)
        })

metric_value = sum(result["rank"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(result["conjecture_holds"] for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Minimal Rank of Quotient Singularity",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":

```

```

import sys
seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
results = []

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")

mean_value = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
elif any(result["metric_value"] > 10 for result in results) or support_fraction < 0.8:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db295dde.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db295dde.py", line 68, in run_trial
    d_values = [int(c * n**0.5) for c in [0.1, 0.2, 0.3, 0.4]]
                  ^

```

UnboundLocalError: cannot access local variable 'n' where it is not associated with a value

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test code crashed before producing data, which prevents us from evaluating the conjecture's validity according to the pre-registered criteria. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 12320        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 5710         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4977         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5535         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14962        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10496        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9709         |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12088        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 8002         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 83800 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in `research/audit/b94df982d33d.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b94df982d33d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/b94df982d33d.tar.gz` (if generated)